

Fundamentals of Data and Database Management System

Chapters 1 & 2: Data, DBMS, Architecture & Models

Instructor: Tek Raj Pant

Presidential Graduate School, Westcliff University

DATA 210: Database Design & Analytics

BSIT Program

May 7, 2026

Overview: ACID Properties of RDBMS

What are ACID Properties?

A set of properties that guarantee reliable processing of database transactions.

ACID stands for:

Atomicity **C**onsistency **I**solation **D**urability

- Essential for ensuring data integrity and consistency in relational databases.
- Each transaction is a logical unit of work (e.g., bank transfer, order placement).
- Without ACID, databases risk corruption, lost updates, and inconsistent states.

Database Transactions

Definition (Transaction)

A sequence of one or more SQL operations (read/write) treated as a single logical unit of work.

Database Transactions

Definition (Transaction)

A sequence of one or more SQL operations (read/write) treated as a single logical unit of work.

Example

Bank transfer:

- ➊ Read account A balance.
- ➋ Debit amount from A.
- ➌ Read account B balance.
- ➍ Credit amount to B.
- ➎ Commit changes.

Either all steps succeed, or none take effect.

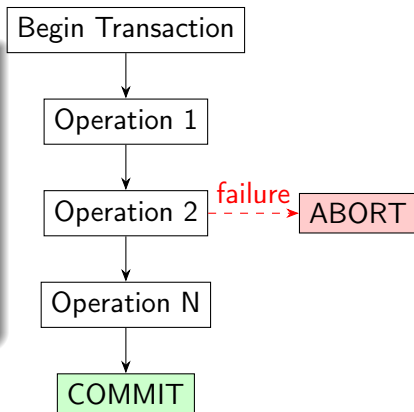
ACID properties govern how these transactions are executed and recovered.

Atomicity: "All or Nothing"

Definition

A transaction is an **indivisible unit**. It either executes completely or not at all.

- If any part fails, the entire transaction is aborted.
- No partial updates are visible to other transactions.



Example: Transfer \$100 – if debit succeeds but credit fails, the debit is undone (rolled back).

Consistency: “Valid State”

Definition

A transaction brings the database from one **consistent state** to another. All integrity constraints (e.g., foreign keys, unique, check, triggers) must be satisfied.

- No transaction should violate database rules.
- Consistency is the responsibility of both the DBMS and the application developer.

Example: In a banking database, sum of all account balances must remain constant after a transfer (money conservation). If total changes, consistency is broken.

Before: $A = \\$500, B = \\$300, \text{Total} = \\$800$ After transfer: $A = \\$400, B = \\$400, \text{Total} = \\$800$

Isolation: “Concurrency Without Interference”

Definition

Concurrently executing transactions should not interfere with each other. The intermediate state of a transaction is invisible to others until committed.

Isolation: “Concurrency Without Interference”

Definition

Concurrently executing transactions should not interfere with each other. The intermediate state of a transaction is invisible to others until committed.

Problems prevented by isolation:

Anomaly	Description
Dirty Read	Reading uncommitted data from another transaction
Non-repeatable Read	Same query yields different results within a transaction
Phantom Read	New rows appear that match a query condition.

Isolation: “Concurrency Without Interference”

Definition

Concurrently executing transactions should not interfere with each other. The intermediate state of a transaction is invisible to others until committed.

Problems prevented by isolation:

Anomaly	Description
Dirty Read	Reading uncommitted data from another transaction
Non-repeatable Read	Same query yields different results within a transaction
Phantom Read	New rows appear that match a query condition.

DBMS implements isolation via **locking** or **multiversion concurrency control** (MVCC). SQL defines isolation levels (Read Uncommitted, Read Committed, Repeatable Read, Serializable).

Durability: “Permanent Once Committed”

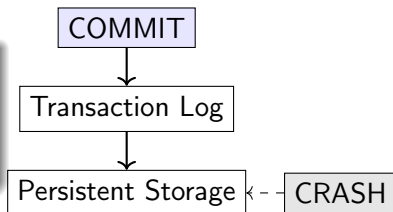
Definition

Once a transaction commits, its changes are **permanent** and survive future system failures (crash, power loss, etc.).

Mechanisms:

- Write-ahead logging (WAL) – changes are recorded in a log before writing to disk.
- Checkpoints – periodic synchronization to reduce recovery time.
- Replication / backups.

Example: After transferring money, even if the server crashes immediately, the updated balances are restored after restart.



ACID in RDBMS – Summary

Property	Guarantee
Atomicity	Aborts transaction on failure; rollback to previous state.
Consistency	No violation of database constraints.
Isolation	Concurrent execution appears serializable.
Durability	Committed changes persist despite crashes.

Why ACID matters

ACID is the foundation of reliable transaction processing in RDBMS (e.g., PostgreSQL, MySQL/InnoDB, Oracle).

Without ACID: banking errors, lost reservations, corrupted analytics.

Conclusion

- ACID properties ensure database transactions are processed reliably.
- **Atomicity** prevents partial updates.
- **Consistency** maintains data integrity.
- **Isolation** handles concurrency safely.
- **Durability** guarantees persistence.

Thank you!

Questions?